llmctl docs/validation

Validation & Verification contract

This project follows an anti-bluff methodology: **every claim about behavior is backed by a command, its exit code, and its raw output** — captured either in the test harness output or in per-profile evidence logs.

The test harness

`tests/run_tests.sh` (invoked by `make test`):

1. Discovers `tests/test_*.sh`.
2. For each test file, prints the exact command (`bash <path>`), runs it as a real subprocess, captures the exit code and the full raw output, and prints both.
3. Builds the summary table **from the actual exit codes** and exits non-zero if any test failed.

Nothing is mocked at the harness level. Determinism on arbitrary machines is achieved with explicit seams:

- `LLMCTL_FAKE_HW=<fixture.json>` — the hardware probe returns the fixture verbatim (after a JSON validity check). Fixtures: `tests/fixtures/hw-baseline.json` (Ryzen 7 2700X / 32 GB / RTX 3060 12 GB / NVMe), `hw-workstation.json` (Threadripper 64-core / 256 GB / 32 GB VRAM), `hw-apple.json` (M4 Max / 64 GB unified), `hw-constrained.json` (8 GB VRAM variant used to exercise eviction protection).
- `LLMCTL_DRY_RUN=1` — service actions (`systemctl`, `launchctl`, `loginctl`) are printed instead of executed. State files (reservations, env files, unit/plist generation) are still real.

- `LLMCTL_HF_BASE=<url>` — the download test serves its fixture from a local `python3 -m http.server` instead of Hugging Face, exercising the real curl/verify/rename code path, including sha256-mismatch rejection.
- All state dirs are relocatable (`LLMCTL_STATE_DIR` etc.) so tests run fully isolated in a `mktemp` directory.

Covered by the harness:

Area	Test
`bash -n` of every shipped script, shebang + strict mode	`test_syntax.sh`
catalog JSON validity, unique ports, required fields, real sha256s	`test_catalog_json.sh`
probe runs on the host, valid JSON, fixture override, failure paths	`test_hardware_probe.sh`
planner tiers, footprints, recommended sets, co-residency groups (exact values, 3 fixtures)	`test_planner.sh`
download + verify + skip + mismatch rejection + evidence log	`test_download.sh`
scheduler start/refuse/evict/protect/switch (dry-run)	`test_scheduler.sh`
systemd unit + launchd plist contents, memory limits from probe	`test_services.sh`
CLI help/version/exit codes/unknown command	`test_cli.sh`

Decision layer (3.1.0), also run by `make test` :

Area	Test
HTTPS gateway, every row of the endpoint inventory, key/401 paths, `--stop` identity checks	`test_gateway_endpoints.sh` , `test_gateway_mutation.sh`

Area	Test
`llmctl decide` shell front end against a real TLS gateway	`test_decide_cli.sh`, `test_decide.sh`
Go unit tests of every `internal/*` package and `cmd/llmctl-decide`	`test_go_unit.sh`
certificates and keys (CLI behaviour, mutations that must be caught)	`test_certs_go_cli.sh`, `test_certs_go_mutation.sh`, `test_keyring_go_cli.sh`, `test_apikey_lifecycle.sh`
registry, dynamic ports, discovery	`test_registry_cli.sh`, `test_registry_discovery.sh`, `test_dynamic_ports.sh`
the 66 defects of the first-candidate register stay fixed	`test_regression_defects.sh` (map: `specs/009-jev-decision-models/evidence/red-to-green-map.json`)
no literal keys in docs/templates; no retired variables; no stray binaries; no secrets in release archives	`test_docs_no_literal_keys.sh`, `test_no_retired_vars.sh`, `test_no_stray_binaries.sh`, `test_release_no_secrets.sh`
every doc reachable from the README	`test_doc_reachability.sh`
no shipped doc or help text calls the enforced validation gate advisory	`test_docs_gate_not_advisory.sh`
rootless vantage container	`test_vantage.sh`, `test_vantage_classifier.sh` (SKIP with a reason when podman or an image is missing)

Per-suite assertion counts are measured by `scripts/doc_counts.sh`, not typed (see `docs/scripts/doc_counts.md`). A suite that cannot run on the host prints `SKIP-SUITE: <reason>` and is reported as SKIP, never PASS. Gate honesty: a decision-model *accuracy* figure comes only from the golden-set runner against a real model (golden-set); none is published yet, and when one is it ships labelled *provisional - labels agent-authored, human review pending*. What the 3.1.0 work did not verify live: limitations.

Release gate status for 3.1.0

- **Manual QA waived by the operator, 2026-10-08; the constitution gate is operator-waived, not satisfied.** The project's constitution (section 11.4.185) makes live manual QA by a person the final gate before a tag. For 3.1.0 the operator waived it, so the release rests on the automated gates listed on this page, the independent code reviews and the full test suite only. This is a recorded waiver, not a passed gate; nothing on this page should be read as evidence that a person exercised the release.
- **Platforms.** Linux is the live-verified platform. macOS is shipped labelled **verified statically only** (launchd, plist and bash 3.2 behaviour were read and fixture-tested, never run on a Mac). Windows is unsupported.
- **Second host.** The second-machine runs used `nezha.local` (ALT Linux, CPU only).
- **Accuracy figures** are provisional: agent-authored labels, human review pending. Per-profile-and-type `experimental` labels (profiles whose measured Wilson lower bound does not clear the baseline) are the shipped policy and are not yet visible in the CLI output; they land with the maturity task, see limitations.

Model validation gate

Contract (operator mandate 2026-10-12, enforced since the 2026-10-14 fix round): **every model llmctl runs must pass a deterministic known-answer suite BEFORE any project, component, service or submodule uses it**; wrong answers are real defects. This is the *model* gate; the harness above

verifies *llmctl*. Implementation: `lib/validate.sh` (front end, state, policy, hooks), `lib/validate_judge.py` (runner, judge, records, stdlib only), the quarantine half of `lib/scheduler.sh` and the unit gate of `lib/svc_hook.sh`. Suites live in `validation/suites/<name>/`; the catalog names each profile's suites in `validation.suites` (every shipped profile names at least one).

Status of the evidence (read this first). Real-model runs exist only from BEFORE this fix round: profile `vision` on this host, 2026-10-13 (vision-live-2026-10-13) and 2026-10-14 (vision-live-2026-10-14); `small` (chat-v1) on this host (small-live-2026-10-14); and a `--host` run against a capped server on the second host (vision-live-nezha-2026-10-14, evidence only, no record). They used the earlier judge (one determinism repeat, no signed records), and every suite file changed since (new canned controls), so **no profile has a current validation record: every profile is NOT VALIDATED until it is validated again** (`llmctl validate-all` lists them). The 26 real answers those runs captured are kept as a regression fixture (`tests/fixtures/validation/recorded_live_answers.json`): the current judge gives every one of them the same verdict. *Never run against a real model: the colibri profiles and every decide- profile** (`validation.executed: false` in the catalog, `execution_status: unexecuted` in `decide-v1`); their suites are proven against fixtures only and their live behaviour is UNCONFIRMED.

Flow

```
flowchart TB
  S["llmctl start / switch / auto / enable / restart / decide scale"] --> K{"fresh, SIGNE
  K -- yes --> PUB["start PUBLIC: documented port, catalog bind host, registry row, unit
  K -- no --> Q["start in QUARANTINE: 127.0.0.1, private ephemeral port, no registry row,
  Q --> V["validate_profile (outside the scheduler lock)"]
  BOOT["systemd boot / Restart= / systemctl start"] --> X{"ExecCondition validated: fresh
  X -- no --> SKIP["unit SKIPPED, exit 77: not failed, not restarted"]
  X -- yes --> WR["waiter: health OK -> validate BEFORE registering"]
  V --> PG{"host pressure gate (PSI / MemAvailable)"}
  PG -- refused --> U["UNDETERMINED exit 2: nothing probed, nothing recorded"]
  PG -- ok --> SD{"suite schema: every check non-empty and known, every probe has PASS+FA
  SD -- defect --> U
  SD -- ok --> B{"fixture files match the sha256 in the suite?"}
  B -- no --> U
```

```
  B -- yes --> C{"judge self-test: canned controls judged as required?"}
  C -- no --> F["FAIL exit 1: the judge is broken"]
  C -- yes --> D{"endpoint serves the expected model id?"}
  D -- "no / unreachable" --> U
  D -- yes --> SM{"the probed PROCESS loads the profile's file? (cmdline, /v1/models path
  SM -- "another file" --> U
  SM -- "same / unconfirmed (flagged)" --> E["every probe x N (LLMCTL_VALIDATE_REPEAT, de
  E --> G["deterministic judge: claims (negation-aware), exact order, single stated numbe
  G --> H["golden-bad controls must FAIL the golden checks; all N answers byte-identical"
  H --> W{"verdict"}
  W -- "wrong answer" --> FAIL["FAIL exit 1"]
  W -- "no answer somewhere" --> U
  W -- "all right" --> P["PASS exit 0"]
  P --> R["record (repeat >= 3 only): real sha256 of every model file = catalog pin, engi
  FAIL --> R2["record: fail (an UNDETERMINED run never replaces a PASS or a FAIL)"]
  R --> PUBL["quarantined -> published (sched_publish); waiter -> registered"]
  FAIL --> STOP["stopped through the normal stop path"]
  U --> STRICT{"LLMCTL_VALIDATE_STRICT=0 AND profile not consumer-facing?"}
  STRICT -- "no (default)" --> STOP
  STRICT -- yes --> UNV["served, marked UNVALIDATED in status; require-validated / endpoi
  PUBL --> RV["llmctl require-validated / llmctl endpoint: 0 / URL only for a fresh signe
```

Gate state machine and exit codes

Source: `diagrams/validation-gate.mmd` (rendered svg). `0` pass, `1` wrong answer, Exit codes in play: 0, 1, 2, 75 (pressure refusal), 77 (unit skipped), 98 (port in use). `2` could not determine (a host-pressure refusal, `rc 75`, is reported as `2`: it is not a verdict). On the systemd path `77` means the unit was skipped for lack of a fresh record, and `98` means the port is already in use (not restarted).

```
stateDiagram-v2
  direction LR
  [*] --> Requested: start / switch / auto / enable / restart / decide scale
  Requested --> Quarantine: no fresh signed PASS record
  Requested --> Published: fresh signed PASS record
  Quarantine --> Refused: host pressure gate refuses (rc 75)
  Refused --> Undetermined: refusal is not a verdict, reported as exit 2
  Quarantine --> Judging: suites run, N repeats, deterministic judge
  Judging --> Pass: every check right (exit 0)
  Judging --> Fail: any wrong answer (exit 1)
  Judging --> Undetermined: no answer, wrong model served, suite defect (exit 2)
```

```
  Pass --> Published: record signed, then published
  Fail --> Stopped: stopped through the normal stop path
  Undetermined --> Stopped: STRICT=1 (default)
  Undetermined --> Unvalidated: STRICT=0 and not consumer-facing
  Published --> [*]: require-validated exit 0, endpoint prints URL
  Stopped --> [*]
  Unvalidated --> [*]: require-validated exit 1
  state "systemd unit path" as Unit {
    [*] --> ExecCondition
    ExecCondition --> UnitSkipped: no fresh record, exit 77
    ExecCondition --> PortCheck: record or live quarantine
    PortCheck --> PortBusy: port in use, exit 98 (not restarted)
    PortCheck --> Waiter: port free
  }
```

Commands and exit codes

Command	Meaning
`llmctl validate <profile> [--host URL] [--json] [--keep]`	run every suite of the profile; a profile that is not running is started UNPUBLISHED (quarantine) and stopped again unless `--keep` (which then says it keeps running)
`llmctl vision validate [--profile vision|vision-pro] ...`	the golden-screenshot suite only
`llmctl validate-all`	every running profile, then a coverage list of EVERY catalog profile: a signed, unexpired PASS record, or `NOT VALIDATED - <reason>` (never-executed suites say so)
`llmctl require-validated <profile>`	exit 0 only for a fresh, signed PASS record that matches what would run now (below); 1 not validated; 2 cannot determine (e.g. `LLMCTL_VALIDATION_MAX_AGE=nan`)
`llmctl endpoint <profile>`	prints `http://127.0.0.1:<port>` ONLY for a running, published, validated profile; exit 1 with the

Command	Meaning
	reason otherwise - **the call a consuming project makes**
`llmctl models verify <profile> [--live]`	checksums, then the suite gate (with `--live`, or when running, the suite is run)
`llmctl models pins`	supply-chain gate: every UNPINNED revision / file sha256 of the catalog; exit 1 while any remains

`0` every check passed; `1` any wrong result (never excused, no threshold is loosened); `2` could not determine (not downloaded, not served, endpoint down, wrong model or file served, truncated or reasoning-only completion, suite defect, pressure refusal, quick mode for a local profile). `1` outranks `2`. UNDETERMINED is never PASS.

Suites and the judge

`chat-v1`, `coder-v1` (text prompt to answer), `vision-golden-v1` (golden screenshots with known content plus two golden-bad control images that must fail the golden checks), `embedding-v1` (cosine ordering plus byte-stable repeated embeddings), `decide-v1` (typed choice questions, below). Sampling is deterministic (temperature 0, fixed seed, top_k 1, no prompt cache). The judge is plain code, never an LLM. Rules (2026-10-14 review, finding 7 - every one has a canned FAIL control in the suites, so the self-test proves it before every real run):

Rules of 2026-10-15 (review finding 4: 42 of 77 adversarial answers were judged wrongly; the reviewer's table is `tests/fixtures/validation/adversarial_answers.json`, 84 rows, judged by `tests/test_validation_judge_adversarial.sh` - 0 wrong verdicts), tightened by the third review of that day (findings 1-5: of 144 NEW adversarial answers 30 clearly wrong ones PASSED and 4 clearly right ones FAILED; the table is `tests/fixtures/validation/adversarial_answers_r3.json`, judged by `tests/test_validation_judge_adversarial_r3.sh` - 0 clear false PASS, 0 clear false FAIL, every refused right answer listed under "Known trade-offs"

below). **Every rule is a positive allow-list of what may stand in an answer, never a list of blocked words (Constitution 11.4.250).**

- **Unicode is folded before any case mapping.** NFKC per character for fullwidth ASCII, every Unicode decimal digit to its ASCII digit (`٣` is 3), typographic apostrophes to `'` , and every OTHER non-ASCII letter becomes a separator - `str.upper()` used to turn the dotless `Paɪs` and the long-s `Pariſ` into PARIS; a Cyrillic `а` was already not an `a` .
- **A claim must be asserted.** `contains_all` / `contains_any` / `forbidden` / `ordered` count a needle only when an occurrence is NOT negated: a negation word (`no` , `not` , `none` , `never` , `without` , `cannot` , `rather` , `instead` , `n't` ...) within the 4 words before it in the same clause (clauses end at a newline, `!` , `?` , `;` , `:` and a `.` that is not a decimal point; `not sure` / `not certain` is a hedge, not a negation of what follows); a denial right after it (`Paris is incorrect` , `Delete is not there` , `X isn't`); a `-less` / `-free` compound (`login-less`); or a later clause that RETRACTS (`Paris? No, Lyon.` , `Three. Wait - four.` , `Actually, ...` : `no` / `wait` / `actually` / `sorry` ... followed by `,` `.` `!` `:` `;` or `-`). `No problem. ORANGE TIGER 42` still claims the text (the retraction comes before it); `OSLO (not PARIS)` and `Paris, not London.` do not claim the second city. `ordered` FAILs when any occurrence is negated (`Not red, green, blue.`). **The word that opens a retraction clause retracts, it does not negate** (review 3, finding 5): in `London. No, Paris.` and `light? no, dark` the `No` withdraws the first answer and the correction after it IS claimed (it used to be read as `no Paris` and FAILed a right answer); `Paris. No, London.` still FAILs (London is claimed). Only a retraction word at the START of a clause after a separator counts: `Lyon, no, Paris.` (`,` is not a clause separator) keeps LYON claimed - see the trade-offs.
- **A parenthetical aside is not the claim** (review 3, finding 3) for `contains_any` : `A dashboard (login screen is next).` does not claim a login screen; an answer that is one aside as a whole (`(Dark)`) is read without its brackets. The screen-type probe additionally `forbid` s the other screen types (`DASHBOARD` , `REGISTRATION` , `SIGN UP` , `SETTINGS` , `HOME SCREEN` / `HOME PAGE` , `CHECKOUT` ; negation-aware as always).

- **Short answers.** `max_words: N` (the "answer with one word" probes): more than N words, or naming alternatives (`or`), FAILs - `Paris or Rome` , `The capital is Lyon (Paris is the largest city).`
- **Only the number.** Every `number_equals` prompt asks for the number only, so the answer must BE a number: every sentence must be `N` (`42` , `42.0` , `58,40` , `$58.40` , `forty-two` , `Three` , `**3**` , a fenced `3` , `(404)`), `N (N')` with the same value (`3 (three)`) or with an allowed word (`404 (Not Found)`), `<arithmetic> = N` (`17 + 25 = 42`) or `<fixed lead-in> N` (`The answer is 42` , `There are three` , `I count 3` , `The sum is 42` ; the lead-ins are a fixed list in `lib/validate_judge.py` `_LEAD_INS`), and all sentences state the same value. **Nothing else may stand next to N except the probe's OWN declared words** (`number_allow` , removed as whole words at the start or end of a sentence): `RED CIRCLES` / `CIRCLES` for the count of red circles (`There are 3 red circles` PASSES), `BLUE SQUARES` / `SQUARES` , `ITEMS` / `FRUITS` for the list count, `USD` / `DOLLARS` for the total, `NOT FOUND` / `HTTP` for HTTP 404, `INVOICE NO` / `INVOICE` / `NO` / `INV` / `#` / `NUMBER` for the invoice. The open `N <any 1-4 words>` form of 2026-10-15 is GONE (review 3, finding 1): `42 thousand` , `42 hundred` , `42 squared` , `42 doubled` , `42 and a half` , `The answer is 42 in hex` , `3 pairs` , `3 million red circles` , `42 I think` , `42 hopefully` , `3 red circles presumably` , `I count 3 I think` name no allowed word and FAIL. A `number_allow` entry ending in `.` declares an abbreviation (`NO.` , `INVOICE NO.` , `INV.`): its period is not a sentence end, so `Invoice No. 7731` and `No. 7731` PASS (they FAILed: the period split the answer into `No` and `7731`), while the period after any other word still ends a sentence (`I count 3 red circles. 4.` states two numbers and FAILs). Not a stated number either: `Not 3` , `There are not 3` , `More than 3` , `At least 3` , `3 or more` , `About 3` , `Maybe 3?` , `3rd` , `3%` , `3/4` , `4.2e1` , `7,731` , `10.000` (three fraction digits: an ambiguous thousands separator), `.42` , `Three. No wait - four.` , `3. 4.` . **Trade-off, stated plainly:** a right number wrapped in other prose (`The answer is 42, not 41.` , `42 kg` , `I see 3 red circles in the image`) FAILs - the prompts ask for the number only.

- **Exact answers.** `exact_text` (the echo probe): the answer, outside one optional fence / quote pair and one trailing `.`, with whitespace collapsed, must equal the value case- and punctuation-exactly (`ZX-4417-B`, `zx_4417` FAIL). `exact_words` (the transcription of a known text): the normalised words must be exactly the expected ones, in order (`TIGER ORANGE 42`, `ORANGE TIGER 42 ZEBRA` FAIL). Since review 3 (finding 2) the list, table and colour probes use it too - `ordered` ignored items that are NOT in its list, so `red green blue yellow`, a hallucinated fifth list item and a hallucinated table row PASSED. A probe may name its own framing PHRASES in `words_allow` (removed only as a whole, contiguous phrase): the colour probe allows `AND`, `FROM LEFT TO RIGHT` and `HERE S THE BREAKDOWN OF THE COLORS` (the latter verbatim from a recorded real answer), so `red, green, and blue` PASSES while `From right to left: red, green, blue` FAILs. A `words_allow` entry may not contain an expected word or a negation word (suite defect). `ordered` remains for suites that want "these in this order, other text allowed".
- **Exact label sets.** `label_set` (review 3, finding 3; the dialog-buttons probe, which asks for "the two labels only"): the answer must name EXACTLY the listed labels, each once, in any order, and nothing else but the probe's `words_allow` (`AND`, `BUTTON`, `BUTTONS`). `Cancel, Delete, OK` (a third label) and `Neither Cancel nor Delete; Save and Close.` FAIL; `Delete and Cancel` PASSES. One clause that DENIES a single other label is a remark, not a label: `Cancel, Delete - no OK button.` PASSES, `Cancel, Delete, not OK but Save` FAILs (more than one word after the denial).
- **JSON as a document.** `json_equals`: the whole answer (one optional ` ```json ` fence) must be exactly ONE JSON document equal to the value - type-exact ( `1` is not `true`, `0` is not `false`, `"1"` is not `1`; `1.0` is `1` ), no extra key, no duplicate key, no trailing text ( `{...} -- no wait: {...}` ), no NaN; a Python dict is not JSON. Also used for the empty-array literal ( `[[]]` and `Not [], it is null.` FAIL, `[ ]` PASSES). `json_subset` keeps its type-exact subset semantics for suites that want it.
- **Code by its syntax tree.** `python_function_sum: add` parses the code of the answer (fenced blocks when present; never executed) and needs exactly one top-level `def add` of two positional parameters whose only `return` is its last statement and returns `p1 + p2` (directly or through one

assigned name), with no control flow or nested definition, no re-bound parameter and no other binding of `add`. A docstring or string literal saying `return a + b`, a dead branch, `return a + b + 1`, the sum in another function and a later `add = lambda ...` FAIL; a comment is not code. Since review 3 (finding 4): the module may hold ONLY the def, imports and a leading docstring, nothing anywhere may CALL anything, and nothing may store through an attribute or a subscript - `add.__code__ = (lambda a, b: a - b).__code__`, `globals()['add'] = ...`, `match 0: case add: ...`, `globals().update(add=None)` inside the body and a module-level `print(add(2, 3))` FAIL. **Trade-off:** a correct `return operator.add(a, b)`, `return sum([a, b])` or `return a.__add__(b)` FAILs (a call), as does a usage example below the function.

- **Known trade-offs (review 3, re-justified).** Right or debatable answers the rules knowingly refuse, each pinned in `adversarial_answers_r3.json` with its justification so a changed verdict is noticed: `42 kg` (an undeclared word next to N); `+42` (a signed number); `42,0` / `58,4` (a one-digit decimal comma is ambiguous with a list); `Lyon, no, Paris.` (a retraction needs a clause separator); `Lyon. I mean Paris.` / `Lyon. Er, Paris.` (not retraction words); `Paris isn't Lyon.` (the post-negation rule reads `X isn't ...` as denying the word before it); `Pa\u200bris` / `42\u200b` (an invisible character is not normalised away); `` `{"color": "blue"}` `` / `` `[]` `` (only a ``` fence is unwrapped); a BOM before JSON; `[].`; `sum([a, b])`, `a.__add__(b)` (calls), a decorated `add`, `a - -b` (no algebra on the AST). Four trade-offs of the round-2 table are resolved by the probe's own allow-lists instead: `USD 58.40`, `404 (Not Found)`, `Not Found (404)` and `HTTP 404` now PASS.
- **The suites changed, so their hashes changed.** The review-3 rules are expressed in the suite files (`number_allow`, `exact_words`, `words_allow`, `label_set`, `forbidden`, new judge controls), so every suite content hash changed: every existing validation record is no longer fresh and every model must be validated again (`llmctl validate <profile>`) before `require-validated` accepts it.
- **No vacuous check.** Empty or unknown checks (`{}`, `contains_all: []`, an uncompilable regex, an item without letters or digits), a probe without a prompt, or a probe without at least one canned PASS **and** one

canned FAIL judge control are a **suite defect**: the run is UNDETERMINED before any request and `selftest` exits 1.

- **N repeats (Constitution 11.4.50).** Every judged probe is asked `LLMCTL_VALIDATE_REPEAT` times (default **3**); all answers must be byte-identical (decide: the choice AND the probabilities). `LLMCTL_VALIDATE_REPEAT=1` is the documented **quick mode**: only the suite's `determinism_repeat` probes are asked once more, the run says `quick mode`, and it is **never recorded** (a local quick PASS exits 2: not validated).
- **Complete answers only:** a completion cut off by the token budget (`finish_reason=length`) with reasoning content, or empty `message.content` while the reasoning channel is non-empty, is UNDETERMINED with a hint to raise `request.max_tokens`. A truncated WRONG answer is a FAIL; a truncated answer that would pass is still UNDETERMINED. Only `message.content` is judged.

decide-v1 (finding 2): five typed choice questions whose answer is defensible BY CONSTRUCTION (the state or the option text states it; the right option is first in two questions and second in three, so an always-A model fails; `invoice_team` is the exact question of the post-download decision smoke). They are asked through the PRODUCTION Go driver of the profile's protocol (`llmctl-decide smoke --question-file F`; letter-logit, nli-onnx, systemone-native) and judged by exact `choice_equals`. A local decide run identifies the model by the probed process's model file (a decide engine has no unauthenticated `/v1/models`): unconfirmed is UNDETERMINED. `decide-tiny` (protocol jev-verdict) has no driver: UNDETERMINED by design. **colibri** reports its family's `default_model_id`, not a file name: the catalog names it in `validation.expect_model_id` (`qwen3.6-colibri`, `glm-5.2-colibri`; source lines in the catalog entry; the qwen36 one cross-checked against the downloaded `config.json`), and a colibri server is expected to load the profile directory.

Records (what `require-validated` checks)

`$LLMCTL_STATE_DIR/validation/<profile>.json` (0600, directory 0700) is written only by `llmctl validate` and holds: status; the **real sha256 of every**

model file with its size; the catalog pins; engine tag; launch fingerprint (exec + arguments minus `--host` / `--port`); per-suite content hash; host id; the model file of the probed process (`served_model`: path, size, mtime, sources, confirmed); `repeat`; the path and **sha256 of this run's own evidence file**; timestamp; and an **HMAC-SHA256 signature** under `$LLMCTL_STATE_DIR/validation/.hmac.key` (0600, created on first use). `require-validated` exits 0 only when the signature verifies, the status is pass, every file hashes to what was recorded AND to its catalog pin, engine/launch/suites/host id match, `repeat >= 3`, the evidence file still hashes to the recorded value, and the record is younger than `LLMCTL_VALIDATION_MAX_AGE` (whole seconds from 0 to 34560000 = 400 days, default 604800; `nan`, `inf`, negative, exponent and larger values exit 2 - a 12-digit value used to make every record fresh for millennia) and not from the future. An older `validation/` directory with looser permissions is tightened to 0700 on use.

- **Hashing.** Model files are hashed by streaming sha256; the result is cached in `.hashcache.json` (0600) keyed by size, mtime_ns, **ctime_ns**, inode and device, each entry HMAC'd - any write changes ctime, so different bytes with a restored size and mtime are re-hashed and refused. Fast path: a cache hit; at record creation every file whose hash came from the cache is hashed again once (the catalog pin is verified once per record creation). A file whose bytes do not match its catalog pin is never recorded as PASS (exit 2). On a 380 GB profile the first key computation and the record step each read the files once.
- **Host identity.** `host_id` = sha256 of a fixed prefix and `/etc/machine-id` (fallback `/var/lib/dbus/machine-id`; last resort the hostname, `host_id_source: hostname`, weaker). The raw machine id is never written; the hostname is informational only - a spoofed hostname changes nothing. **Clone caveat:** a disk image or VM cloned WITH its `/etc/machine-id` (and the state directory) carries valid records to the clone - the host id cannot tell them apart. Regenerate the machine id of a clone (`systemd-machine-id-setup` after removing it) or delete `$LLMCTL_STATE_DIR/validation/` on it; the served-model identity checks (below) still run on every validation.
- **Served model (local runs).** The probed process's model file must be the profile's (command line, `/v1/models`, `/props`); since 2026-10-15

(finding 11) also: its `--mmproj` must lie next to the model file, the model file it MAPS must be the file on disk now (a model replaced after it was loaded is a mismatch), and when the listening process is the env record's engine binary its command line must be the launch of that env record (the record binds that launch). Any mismatch is UNDETERMINED, never a PASS record. A process that does not mmap its model cannot be checked by inode; another program on the port is judged by the model-file sources only.

- **Same-user limit (stated plainly).** The signature stops accidental edits, copied state and hand-written records. A malicious process running as the SAME user can read the key and forge a record; that is out of scope for a file-based gate.
- **Verdict precedence.** A FAIL replaces anything; an UNDETERMINED run never replaces a PASS or a FAIL record (it writes an `undetermined` record only when there is none); a quick-mode run is never recorded; a `--host` run is evidence only.

Enforcement (strict by default)

The gate is no longer advisory - and since 2026-10-14 not even UNDETERMINED is served. A profile is **served** - its documented port, its catalog bind host, a registry row (the decision gateway resolves backends from the registry), an enabled unit - only with a fresh, signed PASS record for what would run. Otherwise:

Path	What happens
`start`, `switch`, `auto` (all through `_sched_start_impl`)	started in **QUARANTINE**: `--host 127.0.0.1` on a private ephemeral port, no registry row, `quarantine=1` in the reservation, `LLMCTL_QUARANTINE_UNTIL` and `LLMCTL_QUARANTINE_VALIDATOR` (the validating llmctl process: pid and start time) in the env record; `validate_after_start` validates it; PASS -> `sched_publish` stops it and starts it published; FAIL/UNDETERMINED -> stopped, exit non-zero. **Every stop of a quarantined instance puts the**

Path	What happens
	env record back as it was before the quarantine start (kept beside it as `<profile>.env.prequarantine`; removed when there was none) - FAIL, UNDETERMINED, a signal, `llmctl stop`, an eviction, a start that never became ready, publication (review 3, finding 6: an ENABLED profile whose quarantine start failed used to keep the quarantine launch as the record its unit boots from). A `switch` whose target fails brings the previous set back through the same gate
`enable`	validated (in quarantine) BEFORE any unit is enabled; FAIL/UNDETERMINED -> nothing enabled
`restart`	with a fresh record: `systemctl restart`; without: stopped, started in quarantine, validated, published or stopped - never restarted in place on its public port
`decide scale`	adding instances needs a fresh record of the base profile (validate it first); scaling down is never blocked
systemd unit (boot, `RestartForce`, `systemctl start`) - **only after `llmctl install`** (see "Upgrading installed units")	`ExecCondition=svc_hook.sh validated %i`: skipped with exit 77 (not failed, not restarted) without a fresh record; a quarantine start passes only while it is unexpired AND its validator process (`LLMCTL_QUARANTINE_VALIDATOR`: same pid, same start time) is alive (review 3, finding 6); an opt-out marker (`LLMCTL_UNVALIDATED`) passes only when the CATALOG says the profile is not consumer-facing - re-read at every start (review 3, finding 9; an unknown profile counts as consumer-facing). The registration waiter validates BEFORE it registers and stops the unit on FAIL/UNDETERMINED; a quarantine unit is never registered. launchd: the same gate in `svc_hook.sh run-engine`, which exits 0 on a refusal and the agent's

Path	What happens
	`KeepAlive` restarts only an unsuccessful exit (`SuccessfulExit=false` ; agents written before 2026-10-15 keep `KeepAlive=true` and retry a refused start every 60 s until they are re-written by `llmctl enable`)
`status` / `status --json`	a `validation:` line / field per running profile: `validated` (only when `require-validated` would accept it now: the record's bytes, engine and launch match), `QUARANTINE (...)` , `UNVALIDATED (<opt-out>)` , `NOT VALIDATED (<reason>)` ; opt-outs in effect are printed as WARN lines
`validate` , `vision validate` , `validate-all` , `models verify` (by hand)	of a SERVED profile (finding 6; every manual entry point): FAIL -> stopped, registry row withdrawn; UNDETERMINED -> keeps running NOT validated (status / `require-validated` / `endpoint` refuse). Of a STOPPED profile: started in quarantine, validated, stopped again, and its previous env record restored (finding 8: an enabled unit used to boot from the quarantine record and was refused). `--host` and the unit waiter's own call never stop anything
`endpoint` , `require-validated`	refuse everything but a running, published, validated profile
llmctld model routes	dispatch to `bin/llmctl start/stop/status` , so the same gate applies

Verdict	Effect
PASS (0)	published (quarantine) / keeps serving; the signed record is written
FAIL (1)	

Verdict	Effect
	stopped through the normal stop path (`sched_stop` -> `svc_stop` ; systemd: `systemctl --user stop <unit>`), re-servation removed, exit non-zero
UNDETER-MINED (2)	**stopped as well** (strict is the default): an unvalidated model is never served. Opt-out below

Opt-outs (documented, loud, never for consumer-facing profiles). `LLMCTL_VALIDATE_STRICT=0` serves an UNDETERMINED profile marked UNVALIDATED (reservation `unvalidated=...` , env record `LLMCTL_UNVALIDATED=...` for the unit gate, a WARN line in `status`) - **refused** for a profile whose catalog `validation.consumer_facing` is not `false` (the default is consumer-facing; every shipped profile is). `LLMCTL_VALIDATE_ON_START=0` (no validation at start) is refused for a consumer-facing profile **that has no fresh record for the launch about to run** - with one it is simply started published (review 2026-10-15 finding 3) - and `switch` evaluates that refusal BEFORE it stops anything (a refused switch leaves the running set untouched; a rollback is never refused, it comes back in quarantine). `LLMCTL_VALIDATE_ON_START=external` starts a profile without a fresh record UNPUBLISHED (quarantine: loopback, private port, no registry row) and leaves its validation to the caller (`scripts/livetest/vision_live_test.sh` validates it with `--host` and stops it); that command never publishes it. Inside the test harness only, `LLMCTL_VALIDATE_ON_START=0` serves unvalidated: `LLMCTL_VALIDATE_HERMETIC=1` **and** the harness file `$LLMCTL_STATE_DIR/.test-harness` written by `tests/helpers.sh` (finding 9: the variable alone - leaked into a production shell - relaxes nothing, is reset with a warning, and `llmctl status` prints `TEST-HARNESS MODE` while the relaxation is active). `LLMCTL_DRY_RUN=1` starts nothing and validates nothing.

Which launch a record vouches for (finding 5). The record binds the launch fingerprint (engine binary + arguments, `--host` / `--port` excluded). `start` and `enable` decide on the launch they are ABOUT to run (`LLMCTL_SEED=7 llmctl start vision` after a PASS without a seed is quarantined and validated again), and the publish step after a quarantine PASS re-checks the record against the launch it publishes (a change in between is refused, not

published). `restart` reuses the unit's env record, so its record is checked against that.

Costs, stated plainly. The first start of a profile without a record loads the model twice (quarantine, then published); later starts with a fresh record load it once. The decide-* profiles cannot be validated on a host without a working `llmctl-decide` that has `--question-file` (rebuild: `llmctl build decide`) and `decide-tiny` cannot be validated at all (it has no validation driver).

Upgrading installed units (review 2026-10-15 finding 1)

Installed units and launchd agents run `lib/svc_hook.sh` straight from the checkout, so updating the tree changes the hooks of units that are already installed and running. The unit-level gates (the `validated` ExecCondition, the `run-engine` gate, the waiter that validates BEFORE it registers) are therefore switched on **only** by the marker `$LLMCTL_STATE_DIR/unit-enforcement` (`validation-gate=1`, mode 0600), which `llmctl install` writes after it regenerated the units; no environment variable can set or clear it.

- **Marker absent** (units installed by an older llmctl): the hooks behave exactly as at c23c2e6 - the waiter registers the unit, then validates only a `kind=llm` profile (FAIL stops it, UNDETERMINED leaves it running NOT validated unless `LLMCTL_VALIDATE_STRICT=1`); a decision unit (`llmctl-onnx@decide-nli`, the decide-* llama units) is registered and never validated or stopped by its own restart, crash or boot. `llmctl status` prints `WARN: the unit-level validation gate is NOT installed`. The CLI gates (`start`, `restart`, `enable`, `switch`, ...) apply regardless: they are what the command you typed does.
- **Marker present**: every row of the table above applies to boot, crash and `Restart=` starts too.

Before running `llmctl install` on a host that serves profiles without a fresh record (today: every profile, see "Status of the evidence"), validate each profile you serve while it runs - `llmctl validate <profile>`, then `llmctl require-validated <profile>` must exit 0 - or accept that its next boot or restart is skipped (exit 77) until `llmctl start <profile>` validates it. Since finding 6 a FAIL of that manual validation STOPS the served profile and

withdraws its registry row (an UNDETERMINED leaves it running): validate at a time a stopped profile is acceptable. A decision profile needs a working `llmctl-decide` with `--question-file` for that (above); a profile that cannot be validated is not served after the install.

Operational safety

- `validate_profile` has ONE stop path for a profile it started (`_val_cleanup_started`): every return after the start (no port reservation, key failure, pressure refusal) stops it unless `--keep` , which says it keeps running. SIGINT/SIGTERM/SIGHUP remove the temp files, kill the judge child by PID (never by process group) and stop such a profile; exit status 130/143/129. A QUARANTINED profile is stopped on such an interrupt too, also when the validator did not start it itself (the post-start validation of `start` / `switch` / `restart` , while it waits for the per-profile validation lock or probes) - a quarantine never outlives an interrupted validation. The scheduler lock is re-entrant within the process that holds it, so a stop from the handler during the quarantine START (lock held) runs inline instead of blocking (review 2026-10-15 finding 2; `tests/test_validate_interrupt.sh`).
- **Host-pressure gate** (`hw_pressure_gate` , lib/hardware.sh): before the validator's own start (footprint included; message "nothing was started or probed") and again before the probe loop (PSI only; message "the probes were NOT sent" - the profile may already be up and is then stopped by the cleanup path). Thresholds: PSI memory `full avg10 > 2` , `some avg10 > 10` , `MemAvailable < footprint + 4096 MiB` . A refusal is UNDETERMINED (exit 2), never a FAIL.
- **RAM budget (finding 10, Constitution 12.6).** Admission uses the tighter of `MemAvailable - 4 GiB` and **60% of MemTotal minus the RAM the running llmctl profiles reserve** (`llmctl plan --json` shows `ram_cap_mb` , `ram_reserved_mb` , `ram_avail_budget_mb`); `LLMCTL_RAM_CAP_PCT` may lower the cap, never raise it. A running llmctl engine unit WITHOUT a reservation (started by systemd itself: boot, `Restart=` , the unit waiter) is counted with its cgroup memory (systemd `MemoryCurrent` ; `ram_units_unreserved_mb`) - review 2026-10-15 finding 10. When systemd cannot report it (`MemoryCurrent` = `infinity` /

`[not set]` / 2^64-1, which used to count **0**), the unit is counted conservatively at its `MemoryMax`, else `MemoryHigh`, else the catalog size of its model files, else a fixed 4096 MiB - never 0 - and `llmctl plan --json` lists it under `budgets.ram_units_estimated` (review 3, finding 7). The hermetic unit list `LLMCTL_FAKE_UNIT_MEM` is honoured only inside the test harness (`validation_hermetic`); outside it it is ignored with a warning. launchd has no per-job cgroup: on macOS only reservations count.

- `LLMCTL_VALIDATE_API_KEY` is sent only to loopback hosts or hosts listed exactly in `LLMCTL_VALIDATE_API_KEY_HOSTS`; redirects are never followed; proxies are ignored; only `http` / `https`; a response is limited to 8 MiB; every request has a wall-clock deadline (`LLMCTL_VALIDATE_TIMEOUT`, finite and > 0); evidence and records are 0600 with userinfo, query and fragment removed from logged URLs.
- **Suites directory.** `LLMCTL_VALIDATION_SUITES_DIR` (and the test-only `LLMCTL_MACHINE_ID_FILE` / `LLMCTL_TEST_HOSTNAME`) are honoured only inside the test harness (`LLMCTL_VALIDATE_HERMETIC=1` and `$LLMCTL_STATE_DIR/.test-harness`).
- Exit-code contract: exit 1 means exactly "the judge printed `VERDICT: FAIL`"; a crash, `die()` or `set -e` failure is UNDETERMINED (which strict mode also stops). Only HTTP 400/422 count as FAIL. The systemd waiter treats only the private FAIL code 64 as a wrong answer and removes the reservation before `systemctl --user stop`.
- Concurrency: every validator of a local profile serialises on `<runtime>/.validate-<profile>.lock` (`LLMCTL_VALIDATE_LOCK_WAIT`, default 900 s): the start hook (`validate_after_start`), the systemd waiter, and - since review 3 (finding 10; they used to take no lock) - the manual `llmctl validate`, `vision validate`, `validate-all` and `models verify`. A lock not obtained in time is UNDETERMINED (exit 2). The lock is re-entrant like the scheduler lock: inline within the process that holds it, and for the waiter's own `llmctl validate` child, which inherits the waiter's descriptor and is named by `LLMCTL_VALIDATE_LOCK_HELD=<profile>:<pid>` (honoured only when that pid is the process itself or an ancestor and, on Linux, the

lock file is really open in it - a forged value is ignored). Lock order everywhere: the validation lock, then the scheduler lock. `--host` takes none (it never touches the local profile). Covered by `tests/test_validation_lock.sh`.

- **The registration waiter needs the port registry.** The systemd-path validation runs in the waiter that `hk_register` spawns; it does nothing when there is no `MAINPID` or the port registry is inactive (`LLMCTL_PORTREG=0|off|no`, no portreg binary, `LLMCTL_DRY_RUN=1`). Such a unit is still gated before it starts by the `validated` ExecCondition (a fresh record is required), it is just not re-validated after it answers. The waiter reads the unit's environment, not your shell.

Supply chain

`llmctl models pins` lists every profile whose `hf_revision` is not a 40-hex commit and every file without a `sha256` (exit 1 while any remains); a download from a moving revision warns `UNPINNED`. Today: 10 revisions (`main`) and 6 colibri config files. None was pinned in this round: no manifest, download record or evidence file names their upstream commit, and a sha256 computed from a local download would only restate what was downloaded; `tests/fixtures/catalog_unpinned_allowlist.txt` is a ratchet - a NEW unpinned entry fails the tests. Unpinned files are still hashed into every validation record, so a changed byte invalidates the record.

Known limits

1. **No live run of the enforcement, the decide-v1 suite or a colibri profile** (host safety: no real profile was started or stopped in this round). Everything in this section is proven against fixtures (recording service manager, fixture OpenAI server, a stub of `llmctl-decide`; the real binary's `--question-file` contract is checked) and is UNCONFIRMED live.
2. **Served-model identity** is established from `/proc` only on Linux and only for processes of the same user; elsewhere it falls back to `/v1/models` / `/props` paths and is otherwise flagged UNCONFIRMED (a PASS is still recorded for an OpenAI-compatible engine; for a decide engine it is UNDETERMINED).

3. `test_livetest_watchdog.sh` does not finish within the 580 s bound used in this round (neither at the previous head; a duration, not a failure).

Tests of the gate

Area	Test
judge soundness: hedged numbers, order, negation, comments, empty checks, typed JSON, controls coverage, N=3, real recorded answers	`test_validation_judge_soundness.sh`
signed records, real file hashes, evidence binding, machine id, finite max age, verdict precedence, quick mode	`test_validation_record_integrity.sh`
the model file of the probed process	`test_validation_served_identity.sh`
one cleanup path, pressure-gate wording	`test_validation_cleanup.sh`
decide-v1, colibri ids, coverage in validate-all, the real `--question-file`	`test_validation_decide_colibri.sh`
enforcement: quarantine, publish, strict default, opt-outs, enable/restart/scale/switch, unit ExecCondition, waiter, endpoint, status	`test_validation_enforcement.sh`
unpinned gate, 60% RAM cap	`test_supply_chain_ramcap.sh`
mutants of the new branches, each caught (68; run it per	`test_validation_mutants.sh`

Area	Test
`MUTANT_GROUPS` group - see below)	
judge, controls, mutations, state/freshness, start hook against the fixture server	`test_vision_validate.sh`
golden fixture images and the expectation file	`test_vision_golden_fixtures.sh`
every suite: structure, determinism, controls, per-probe discrimination	`test_validation_suites.sh`
the wiring of the gate (start/switch/auto, restart/enable, systemd waiter, pressure)	`test_gate_wiring_start_paths.sh`, `test_gate_wiring_restart_enable.sh`, `test_gate_wiring_systemd_waiter.sh`, `test_gate_wiring_pressure.sh`, `test_gate_waiter_stop.sh`
the defect register of the first independent review	`test_validation_gate_defects.sh`, `test_validation_exit_mapping.sh`, `test_validation_http_codes.sh`, `test_validation_identity.sh`, `test_validation_lock.sh`
review 2026-10-15: verdict policy of every manual entry point, env record restored (6, 8), unit RAM (10), served launch / mmproj / inode, max age, status label (11)	`test_validate_policy.sh`, `test_validate_policy_entrypoints.sh`, `test_ramcap_units.sh`, `test_validation_served_launch.sh`, `test_validation_r2_small.sh`

Running the mutation harness. `tests/test_validation_mutants.sh` holds 68 mutants (review 3 of 2026-10-15 added 22) and takes ~1180 s in total (measured 2026-10-15), more than one bounded run (`bounded-run -T 580`): a full run is cut off by the bound (rc 124), which is a duration cut-off, not an assertion failure. Run it one group per bounded run - `MUTANT_GROUPS=<group>`

`bash tests/test_validation_mutants.sh` for each of `judge judge2 judge3 suites record served cleanup enforcement qenv vlock marker interrupt gate policy ramunits ramcap`. Measured per group (s): judge 31, judge2 4, judge3 7, suites 10, record 178, served 14, cleanup 3, enforcement 217, qenv 149, vlock 45, marker 68, interrupt 166, gate 211, policy 57, ramunits 15, ramcap 7 - every group well inside the bound. The Go-side mutants of the gateway are in `tests/test_gateway_mutation.sh` (40 assertions, 408 s).

Vision tiers and model identity (Constitution §11.4.278)

HelixQA's UI QA (Constitution §11.4.278) uses a Vision model as its eyes, and that model must hold a fresh validation record under §11.4.277 first. llmctl ships two Vision profiles that share the golden-screenshot suite `vision-golden-v1`: `vision` (hardware tier `baseline`, port 8082) and `vision-pro` (tier `workstation`, port 8083). A consumer declares an ordered ladder of at least two validated tiers; a tier without a fresh signed PASS (`llmctl require-validated <profile>` under `LLMCTL_VALIDATE_STRICT=1`) makes its checks `not_validated`, never PASS. Every verdict and per-frame record must name the model identity: profile, tier, model and `mmproj` sha256, quantisation, engine and version, launch fingerprint, and the validation-record id and age (the validation record carries these inputs; see the record fields above). A lower-tier PASS is never presented as a higher-tier PASS.

Source: `diagrams/vision-tiers.mmd` (rendered svg).

```
flowchart TB
  P["HelixQA / consumer needs eyes (Constitution 11.4.278)"] --> L{"declared ladder of va
  L --> T1["tier 1: profile vision (baseline hardware, port 8082)"]
  L --> T2["tier 2: profile vision-pro (workstation hardware, port 8083)"]
  T1 --> V1{"llmctl require-validated vision under STRICT, exit 0?"}
  T2 --> V2{"llmctl require-validated vision-pro under STRICT, exit 0?"}
  V1 -- "no: exit 1 or 2" --> NV["checks are not_validated, no PASS is issued"]
  V2 -- "no: exit 1 or 2" --> NV
  V1 -- yes --> ID1["verdict carries model identity: profile, tier, model and mmproj sha2
  V2 -- yes --> ID2["verdict carries the same identity fields"]
```

```
  ID1 --> OUT["per-frame record: a lower-tier PASS is never shown as a higher-tier PASS"]
  ID2 --> OUT
```

Evidence boundary (stated plainly). Real-model Vision runs exist for `vision` only (one host, 21 pass / 0 fail / 0 undetermined each); `vision-pro` has no real-model run in this repository, and no profile currently holds a fresh record (see the status note above). Whether the consuming HelixQA code reads these identity fields is the consumer's obligation and is not verified here.

Constitution mapping (hub engine)

llmctl is a constitution hub engine (`constitution/submodules/llmctl` ; Constitution §11.4.28(C) and §11.4.31, `dependency-layout.md`). The gate in this document is its implementation of §11.4.277: UNDETERMINED is never a pass; decision-bearing use runs with `LLMCTL_VALIDATE_STRICT=1` and gates on `llmctl require-validated <profile>` ; models come only through llmctl. The decisions-model layer (§11.4.279) rides on the same gate: the decide gateway serves only validated profiles (decide-gateway). The claude_toolkit JEV layer consumes the gateway; it does not replace the gate.

Model download verification

`llmctl models download <profile>` :

1. Skips files that exist **and** match the catalog sha256.
2. Downloads with `curl -L --continue-at -` (resumable) to `<file>.part` .
3. Verifies size **and** sha256 of the `.part` file; a mismatch is a hard failure and the content never reaches the final path.
4. Atomically renames into place only after verification.
5. GGUF profiles: boots the real `llama-server` (`--ctx-size 512 --n-gpu-layers 0`), polls `/health` , sends the deterministic prompt "Reply with exactly: OK" at temperature 0, and requires "OK" in the response. Colibri profiles: `coli doctor` when installed, else a structural check (non-empty safetensors shards + config).

6. Appends command + exit code + output evidence to `~/.local/state/llmctl/verify/<profile>.log`.

Catalog checksums were captured from the Hugging Face API (`/api/models/<repo>?blobs=true`, the `lfs.sha256` of each blob) at generation time. Entries whose sha256 could not be captured carry `"sha256": null` (non-LFS config files); for those the checksum is fetched live from the same API at download time, and the download **fails hard** if it cannot be obtained. `llmctl models pins` lists them and every moving `hf_revision` (see "Supply chain" above).

What is NOT covered

- **No GPU in the test sandbox.** CUDA/ROCm/Metal build paths and GPU-offload runtime behavior are validated by code inspection and by the planner tests against fixtures, not by executing on real GPUs.
- **No systemd/launchd in the sandbox.** Service lifecycle is tested via `LLMCTL_DRY_RUN=1`; unit/plist *contents* are asserted exactly, but the units were not executed under a real systemd user session here.
- **Engine compilation is not run in CI/sandbox** (no toolchain weight budget). `llmctl build` verifies the produced binaries with `--version` when actually executed on a real machine.
- **The smoke test requires a downloaded multi-GB model**; it is exercised in the harness only in disabled form (`LLMCTL_SMOKE=0`).
- Model quality/perplexity is out of scope; verification is integrity + "serves a deterministic completion", not benchmark scores.